FastAPI

Hands-on, Step-by-Step

From venv setup to GenAI apps with GPT-4o-mini and embeddings.

13 phases · runnable code at every step · build real AI apps

Work through it in order. Type every line. Run every example.

How to use this guide

This guide takes you from an empty folder to a working GenAI backend in thirteen phases. Each phase builds on the previous one — do not skip ahead. Every code block is meant to be typed (or pasted) and run.

What you need before starting:

- **Python 3.10 or newer.** Check with python3 --version.
- **A code editor.** VS Code is recommended.
- **A terminal.** Terminal.app on Mac, PowerShell or Windows Terminal on Windows.
- **An OpenAI API key** (only needed from Phase 11 onwards — for GenAI features).

Convention used in this guide
Lines starting with a $ symbol are shell commands you type in the terminal. Code blocks without $ are Python files you save and run. Anything in a dark terminal box is what you should see on screen.

Phase 0 — Environment Setup with venv

Before any FastAPI code, set up an isolated Python environment. This keeps your project dependencies separate from your system Python — essential to avoid version conflicts later.

Step 1. Create the project folder

```
$ mkdir fastapi-demo
$ cd fastapi-demo
```

Step 2. Create a virtual environment

On Mac and Linux:

```
$ python3 -m venv venv
```

On Windows:

```
> python -m venv venv
```

> **Be patient**
> The venv command takes 10–30 seconds. Do NOT press Ctrl+C while it runs — if you interrupt it, you'll get a half-built venv folder and activation will fail. If that happens, delete the folder with rm -rf venv and run the command again.

Step 3. Activate the venv

Mac and Linux:

```
$ source venv/bin/activate
```

Windows (PowerShell):

```
> venv\Scripts\activate
```

Your prompt now shows (venv) at the start — that's how you know it's active:

```
(venv) yourname@machine fastapi-demo %
```

Step 4. Install FastAPI and Uvicorn

```
(venv) $ pip install fastapi uvicorn
```

Step 5. Save your dependencies

Always pin your dependencies to a requirements file so others (and future-you) can reproduce the environment.

```
(venv) $ pip freeze > requirements.txt
```

Later, to recreate the environment elsewhere:

```
(venv) $ pip install -r requirements.txt
```

Useful commands to remember

Command	What it does
source venv/bin/activate	Activate venv (Mac/Linux)
venv\Scripts\activate	Activate venv (Windows)
deactivate	Exit the venv
pip list	Show installed packages
pip freeze > requirements.txt	Save dependency list
pip install -r requirements.txt	Restore dependencies
rm -rf venv	Delete a broken venv (Mac/Linux)

Phase 1 — Foundations: Your First API

What is an API?

An API (Application Programming Interface) is a contract: a set of URLs that accept requests and return responses. The frontend (or another service) calls these URLs to read or change data.

FastAPI is a Python framework that makes building APIs fast — both in terms of the developer (less code) and the runtime (high performance).

HTTP Methods — the verbs of the web

Method	Used for
GET	Read data. Doesn't change anything. Example: fetch a user.
POST	Create something new or trigger an action. Example: register a user.
PUT	Replace a resource completely.
PATCH	Update part of a resource.
DELETE	Remove a resource.

Status codes you'll see most

Code	Meaning
200 OK	Success.
201 Created	New resource created.
400 Bad Request	Client sent invalid data.
401 Unauthorized	Missing or invalid auth.
404 Not Found	Resource doesn't exist.
422 Unprocessable	Validation failed (FastAPI's default for bad input).
500 Internal Server Error	Your code crashed.

Your first FastAPI app

Create a file called main.py inside fastapi-demo/:

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello FastAPI"}
```

Run it:

```
(venv) $ uvicorn main:app --reload
```

Three URLs will now work in your browser:

- http://127.0.0.1:8000/ → your API
- http://127.0.0.1:8000/docs → interactive Swagger UI
- http://127.0.0.1:8000/redoc → alternative ReDoc UI

Multiple endpoints

Add more routes. Each is just a Python function with a decorator.

```
# main.py
from fastapi import FastAPI

app = FastAPI(title="My First API", version="0.1.0")

@app.get("/")
def home():
    return {"message": "Hello FastAPI"}

@app.get("/about")
def about():
    return {"course": "FastAPI Basics", "author": "You"}

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

What --reload does

Uvicorn watches your files and restarts the server every time you save. This is great for development but slow and unsafe for production — never use --reload on a deployed server.

Phase 2 — Path and Query Parameters

APIs need to accept input. The two simplest ways are through the URL itself: path parameters (part of the path) and query parameters (after a ? in the URL).

Path parameters

Use curly braces in the path. FastAPI converts the value to the type you declare.

```
# main.py (add to existing file)

@app.get("/user/{name}")
def get_user(name: str):
    return {"user": name}

@app.get("/items/{item_id}")
def get_item(item_id: int):
    return {"item_id": item_id, "type": type(item_id).__name__}
```

Try it:

```
GET http://127.0.0.1:8000/user/Ahmed
→ {"user": "Ahmed"}

GET http://127.0.0.1:8000/items/42
→ {"item_id": 42, "type": "int"}

GET http://127.0.0.1:8000/items/abc
→ 422 Unprocessable Entity   (validation error — abc isn't an int)
```

Query parameters

Any function argument not in the path becomes a query parameter.

```
@app.get("/search")
def search(name: str, limit: int = 10):
    return {"query": name, "limit": limit}
```

Try it:

```
GET http://127.0.0.1:8000/search?name=Ahmed&limit=5
→ {"query": "Ahmed", "limit": 5}

GET http://127.0.0.1:8000/search?name=Ahmed
→ {"query": "Ahmed", "limit": 10}    (uses default)
```

Optional query parameters

```
from typing import Optional

@app.get("/products")
def list_products(category: Optional[str] = None, in_stock: bool = True):
    return {
        "category": category or "all",
        "in_stock": in_stock
    }
```

Pagination basics

```
fake_db = [{"id": i, "name": f"Item {i}"} for i in range(1, 101)]

@app.get("/products/page")
def paginated(skip: int = 0, limit: int = 10):
    return {
        "total": len(fake_db),
        "items": fake_db[skip : skip + limit]
    }
```

Type hints = validation, for free
Because limit is annotated as int, FastAPI will reject /products/page?limit=abc with a clear 422 error. You write zero validation code.

Phase 3 — Request Body and Pydantic

For anything more complex than a few URL parameters, you need a request body. The body is JSON sent in a POST or PUT request. FastAPI uses Pydantic models to validate it.

Your first Pydantic model

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel, Field
from typing import Optional

app = FastAPI()

class User(BaseModel):
    name: str = Field(min_length=2, max_length=50)
    email: str
    age: int = Field(ge=0, le=120)
    bio: Optional[str] = None

@app.post("/register")
def register(user: User):
    return {"status": "registered", "user": user}
```

Test it from /docs — there's now a Try it out button. Or use curl:

```
$ curl -X POST http://127.0.0.1:8000/register \
    -H "Content-Type: application/json" \
    -d '{"name":"Ahmed","email":"a@b.com","age":28}'

→ {"status":"registered","user":{"name":"Ahmed","email":"a@b.com","age":28,"bio":null}}
```

Field types and validation

Field(...) option	What it does
min_length / max_length	String length bounds
ge / le	Number bounds (greater-equal, less-equal)
gt / lt	Strict bounds

Field(...) option	What it does
default=...	Default value when field is missing
regex / pattern	Regex validation
description='...'	Shows up in /docs

Nested models

```
class Address(BaseModel):
    street: str
    city: str
    country: str = "Pakistan"

class Customer(BaseModel):
    name: str
    email: str
    address: Address
    tags: list[str] = []

@app.post("/customers")
def create_customer(customer: Customer):
    return customer
```

A complete Todo example

```
from pydantic import BaseModel, Field
from datetime import datetime

class Todo(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    description: Optional[str] = None
    completed: bool = False
    priority: int = Field(default=1, ge=1, le=5)
    due_date: Optional[datetime] = None

@app.post("/todos")
def create_todo(todo: Todo):
    return {"created": True, "todo": todo}
```

Phase 4 — Building a Full CRUD API

CRUD stands for Create, Read, Update, Delete — the four basic operations of almost every API. Here we'll build a complete Notes API using a Python dict as a fake database. You'll swap in a real database in Phase 7.

The complete notes API

Create a new file or replace main.py with this:

```
# main.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime

app = FastAPI(title="Notes API")

# ---- Models ----
class NoteCreate(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    content: str

class NoteUpdate(BaseModel):
    title: Optional[str] = None
    content: Optional[str] = None

class Note(BaseModel):
    id: int
    title: str
    content: str
    created_at: datetime

# ---- Fake DB ----
notes_db: dict[int, Note] = {}
next_id = 1

# ---- CREATE ----
@app.post("/notes", status_code=201, response_model=Note)
def create_note(payload: NoteCreate):
    global next_id
    note = Note(
        id=next_id,
        title=payload.title,
```

```
        content=payload.content,
        created_at=datetime.utcnow(),
    )
    notes_db[next_id] = note
    next_id += 1
    return note

# ---- READ (list) ----
@app.get("/notes", response_model=list[Note])
def list_notes(skip: int = 0, limit: int = 20):
    return list(notes_db.values())[skip : skip + limit]

# ---- READ (one) ----
@app.get("/notes/{note_id}", response_model=Note)
def get_note(note_id: int):
    if note_id not in notes_db:
        raise HTTPException(404, detail="Note not found")
    return notes_db[note_id]

# ---- UPDATE ----
@app.patch("/notes/{note_id}", response_model=Note)
def update_note(note_id: int, payload: NoteUpdate):
    if note_id not in notes_db:
        raise HTTPException(404, detail="Note not found")
    note = notes_db[note_id]
    if payload.title is not None:
        note.title = payload.title
    if payload.content is not None:
        note.content = payload.content
    return note

# ---- DELETE ----
@app.delete("/notes/{note_id}", status_code=204)
def delete_note(note_id: int):
    if note_id not in notes_db:
        raise HTTPException(404, detail="Note not found")
    del notes_db[note_id]
```

Exercise the API

```
# Create
$ curl -X POST http://127.0.0.1:8000/notes \
    -H "Content-Type: application/json" \
    -d '{"title":"Buy milk","content":"2 liters"}'
```

```
# List
$ curl http://127.0.0.1:8000/notes

# Get one
$ curl http://127.0.0.1:8000/notes/1

# Update
$ curl -X PATCH http://127.0.0.1:8000/notes/1 \
    -H "Content-Type: application/json" \
    -d '{"title":"Buy oat milk"}'

# Delete
$ curl -X DELETE http://127.0.0.1:8000/notes/1
```

What you just learned

HTTPException for proper error responses, status_code for HTTP semantics, response_model to shape the output, and the difference between PATCH (partial update) and PUT (full replace). This is the foundation every REST API uses.

Phase 5 — Response Models

Response models do three jobs at once: they validate what your endpoint returns, they shape the JSON output, and they hide sensitive fields from leaking to the client.

Hiding sensitive fields

Suppose your internal user model has a password hash. You never want to return it. Use a separate response model.

```
from pydantic import BaseModel, EmailStr

# Internal model — used inside the app
class UserInDB(BaseModel):
    id: int
    email: EmailStr
    name: str
    hashed_password: str   # never expose this

# Public-facing model — what the client sees
class UserPublic(BaseModel):
    id: int
    email: EmailStr
    name: str

@app.get("/me", response_model=UserPublic)
def get_me() -> UserInDB:
    # Even though we return UserInDB, the response_model filters
    # the output and hashed_password never reaches the client.
    return UserInDB(id=1, email="a@b.com", name="Ahmed", hashed_password="secret")
```

Custom status codes and responses

```
from fastapi import status
from fastapi.responses import JSONResponse

@app.post("/orders", status_code=status.HTTP_201_CREATED)
def create_order():
    return {"order_id": 123}

@app.get("/maintenance")
```

```
def maintenance():
    return JSONResponse(
        status_code=503,
        content={"error": "under_maintenance", "retry_after": 300}
    )
```

response_model_exclude_unset

When you only want to return fields that were explicitly set (rather than defaults), use response_model_exclude_unset=True. Useful for PATCH responses.

```
@app.patch("/users/{uid}", response_model=User, response_model_exclude_unset=True)
def patch_user(uid: int, payload: User): ...
```

Phase 6 — Scalable Project Structure

A single main.py file is fine for learning. For anything real, split your app into layers: routers (URL handlers), services (business logic), models (data), and config (settings).

Recommended folder layout

```
fastapi-demo/
├── venv/
├── .env                    # secrets — never commit
├── .gitignore
├── requirements.txt
├── app/
│   ├── __init__.py
│   ├── main.py             # FastAPI() + include routers
│   ├── config.py           # settings, env vars
│   ├── deps.py             # shared dependencies
│   ├── routers/
│   │   ├── __init__.py
│   │   ├── notes.py
│   │   └── users.py
│   ├── schemas/            # Pydantic models
│   │   ├── __init__.py
│   │   └── note.py
│   └── services/           # business logic, db queries, llm calls
│       ├── __init__.py
│       └── note_service.py
└── tests/
```

Settings with pydantic-settings

```
(venv) $ pip install pydantic-settings python-dotenv
```

Create a .env file at the project root:

```
# .env
APP_NAME=Notes API
DEBUG=true
DATABASE_URL=sqlite:///./notes.db
OPENAI_API_KEY=sk-...
```

Never commit .env
Add .env to your .gitignore immediately. API keys committed to GitHub are scraped within minutes and your account can be billed thousands of dollars.

Load it from app/config.py:

```
# app/config.py
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_name: str = "FastAPI App"
    debug: bool = False
    database_url: str = "sqlite:///./app.db"
    openai_api_key: str = ""

    model_config = SettingsConfigDict(env_file=".env")

settings = Settings()
```

Using a router

```
# app/routers/notes.py
from fastapi import APIRouter, HTTPException
from app.schemas.note import Note, NoteCreate

router = APIRouter(prefix="/notes", tags=["notes"])

@router.get("")
def list_notes():
    return []

@router.post("", status_code=201)
def create_note(payload: NoteCreate):
    return {"id": 1, "title": payload.title}
```

```
# app/main.py
from fastapi import FastAPI
from app.config import settings
from app.routers import notes

app = FastAPI(title=settings.app_name, debug=settings.debug)
```

```
app.include_router(notes.router)
```

Run from the project root:

```
(venv) $ uvicorn app.main:app --reload
```

Phase 7 — Database Integration with SQLAlchemy

Time to make data persistent. We'll use SQLAlchemy (the standard Python ORM) and SQLite (a file-based database — zero setup).

Install

```
(venv) $ pip install sqlalchemy
```

Database setup

```
# app/database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from app.config import settings

engine = create_engine(
    settings.database_url,
    connect_args={"check_same_thread": False}  # SQLite only
)
SessionLocal = sessionmaker(bind=engine, autoflush=False)
Base = declarative_base()

def get_db():
    """Dependency: gives each request its own DB session."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Define a model

```
# app/models.py
from sqlalchemy import Column, Integer, String, DateTime, Boolean
from datetime import datetime
from app.database import Base
```

```
class Note(Base):
    __tablename__ = "notes"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(200), nullable=False)
    content = Column(String, nullable=False)
    completed = Column(Boolean, default=False)
    created_at = Column(DateTime, default=datetime.utcnow)
```

Pydantic schemas (separate from DB models)

```
# app/schemas/note.py
from pydantic import BaseModel
from datetime import datetime

class NoteCreate(BaseModel):
    title: str
    content: str

class NoteRead(BaseModel):
    id: int
    title: str
    content: str
    completed: bool
    created_at: datetime

    class Config:
        from_attributes = True   # allows reading SQLAlchemy objects
```

Router using the database

```
# app/routers/notes.py
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from app.database import get_db
from app import models
from app.schemas.note import NoteCreate, NoteRead

router = APIRouter(prefix="/notes", tags=["notes"])

@router.post("", response_model=NoteRead, status_code=201)
def create(payload: NoteCreate, db: Session = Depends(get_db)):
    note = models.Note(title=payload.title, content=payload.content)
```

```
    db.add(note)
    db.commit()
    db.refresh(note)
    return note

@router.get("", response_model=list[NoteRead])
def list_all(db: Session = Depends(get_db)):
    return db.query(models.Note).all()

@router.get("/{nid}", response_model=NoteRead)
def get_one(nid: int, db: Session = Depends(get_db)):
    note = db.query(models.Note).get(nid)
    if not note:
        raise HTTPException(404, "Note not found")
    return note

@router.delete("/{nid}", status_code=204)
def remove(nid: int, db: Session = Depends(get_db)):
    note = db.query(models.Note).get(nid)
    if not note:
        raise HTTPException(404, "Note not found")
    db.delete(note)
    db.commit()
```

Create the tables at startup

```
# app/main.py
from fastapi import FastAPI
from app.database import Base, engine
from app import models  # noqa — ensures model is imported
from app.routers import notes

Base.metadata.create_all(bind=engine)

app = FastAPI()
app.include_router(notes.router)
```

Production note
create_all is fine for development. For real apps, use a migration tool like Alembic. It tracks every schema change, so you can update production DBs safely without losing data.

Phase 8 — Authentication with JWT

Most APIs need to know who is calling. We'll use bcrypt to hash passwords and JWT (JSON Web Tokens) to identify users on subsequent requests.

Install

```
(venv) $ pip install "passlib[bcrypt]" "python-jose[cryptography]" python-multipart
```

Password hashing

```
# app/security.py
from passlib.context import CryptContext
from datetime import datetime, timedelta
from jose import jwt
from app.config import settings

pwd = CryptContext(schemes=["bcrypt"], deprecated="auto")

SECRET_KEY = "change-me-and-keep-in-env"
ALGORITHM = "HS256"
EXPIRES_MIN = 60

def hash_password(plain: str) -> str:
    return pwd.hash(plain)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd.verify(plain, hashed)

def create_token(user_id: int) -> str:
    payload = {
        "sub": str(user_id),
        "exp": datetime.utcnow() + timedelta(minutes=EXPIRES_MIN),
    }
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

def decode_token(token: str) -> int:
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    return int(payload["sub"])
```

User model and signup/login routes

```
# app/models.py — add
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    email = Column(String, unique=True, index=True, nullable=False)
    hashed_password = Column(String, nullable=False)
```

```
# app/routers/auth.py
from fastapi import APIRouter, Depends, HTTPException
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.orm import Session
from pydantic import BaseModel, EmailStr
from app.database import get_db
from app import models
from app.security import hash_password, verify_password, create_token

router = APIRouter(prefix="/auth", tags=["auth"])

class SignupIn(BaseModel):
    email: EmailStr
    password: str

@router.post("/signup")
def signup(payload: SignupIn, db: Session = Depends(get_db)):
    if db.query(models.User).filter_by(email=payload.email).first():
        raise HTTPException(400, "Email already registered")
    user = models.User(
        email=payload.email,
        hashed_password=hash_password(payload.password),
    )
    db.add(user); db.commit(); db.refresh(user)
    return {"id": user.id, "email": user.email}

@router.post("/login")
def login(form: OAuth2PasswordRequestForm = Depends(),
          db: Session = Depends(get_db)):
    user = db.query(models.User).filter_by(email=form.username).first()
    if not user or not verify_password(form.password, user.hashed_password):
        raise HTTPException(401, "Invalid credentials")
    return {"access_token": create_token(user.id), "token_type": "bearer"}
```

Protecting routes

```
# app/deps.py
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session
from app.database import get_db
from app import models
from app.security import decode_token

oauth2 = OAuth2PasswordBearer(tokenUrl="/auth/login")

def get_current_user(token: str = Depends(oauth2),
                     db: Session = Depends(get_db)) -> models.User:
    try:
        user_id = decode_token(token)
    except Exception:
        raise HTTPException(401, "Invalid token")
    user = db.query(models.User).get(user_id)
    if not user:
        raise HTTPException(401, "User not found")
    return user

# Now use it on any protected endpoint:
# @router.get("/me")
# def me(user: models.User = Depends(get_current_user)):
#     return {"id": user.id, "email": user.email}
```

Test from /docs

When you add OAuth2PasswordBearer, Swagger UI gets an Authorize button. Click it, log in once, and every Try it out call automatically includes your token.

Phase 9 — Async FastAPI

Async is what makes FastAPI fast for I/O-heavy workloads — calling external APIs, querying a database, talking to an LLM. The pattern is simple: declare async def, then use await for anything that takes time.

Sync vs async — when to choose what

You're doing	Use
Calling an LLM or external HTTP API	async def + await
Awaiting an async DB driver (asyncpg, motor)	async def + await
Heavy CPU work (parsing PDFs, image processing)	def (sync) — and consider a worker queue
A blocking SDK with no async version	Wrap with await asyncio.to_thread(...)

Async endpoints in practice

```
import asyncio
import httpx

@app.get("/weather/{city}")
async def weather(city: str):
    async with httpx.AsyncClient() as client:
        r = await client.get(f"https://wttr.in/{city}?format=j1")
        return r.json()
```

Running things in parallel

If a request needs two independent operations, run them concurrently with asyncio.gather:

```
import asyncio

@app.get("/dashboard/{user_id}")
async def dashboard(user_id: int):
    profile, posts, notifications = await asyncio.gather(
```

```
        fetch_profile(user_id),
        fetch_posts(user_id),
        fetch_notifications(user_id),
    )
    return {"profile": profile, "posts": posts, "notifications": notifications}
```

Common mistake

Calling a blocking function (e.g. time.sleep, requests.get, a sync DB driver) inside an async endpoint blocks the entire server. Use the async version of the library, or wrap the blocking call in asyncio.to_thread.

Phase 10 — Middleware, CORS, Files, and Background Tasks

CORS — required for frontend integration

If your React or Vue app runs on a different origin than your FastAPI server (almost always true in development), the browser will block the requests unless your API explicitly allows them.

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",   # Vite
        "http://localhost:3000",   # Next.js / CRA
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Custom middleware

```
import time
from uuid import uuid4

@app.middleware("http")
async def add_timing(request, call_next):
    request.state.rid = uuid4().hex[:8]
    start = time.perf_counter()
    response = await call_next(request)
    elapsed = time.perf_counter() - start
    response.headers["X-Request-ID"] = request.state.rid
    response.headers["X-Process-Time"] = f"{elapsed:.3f}"
    return response
```

File upload

```
from fastapi import UploadFile, File
```

```
@app.post("/upload")
async def upload(file: UploadFile = File(...)):
    contents = await file.read()
    with open(f"uploads/{file.filename}", "wb") as f:
        f.write(contents)
    return {
        "filename": file.filename,
        "size": len(contents),
        "content_type": file.content_type
    }
```

Multiple files

```
@app.post("/upload-many")
async def upload_many(files: list[UploadFile] = File(...)):
    return [{"name": f.filename, "size": len(await f.read())} for f in files]
```

Background tasks

Run something after sending the response — sending an email, logging, updating an index.

```
from fastapi import BackgroundTasks

def write_log(message: str):
    with open("log.txt", "a") as f:
        f.write(message + "\n")

@app.post("/contact")
def contact(msg: str, bg: BackgroundTasks):
    bg.add_task(write_log, f"contact: {msg}")
    return {"status": "received"}
```

Cookies and headers

```
from fastapi import Cookie, Header
from fastapi.responses import JSONResponse

@app.get("/whoami")
def whoami(
    user_agent: str = Header(default=None),
```

```
    session_id: str | None = Cookie(default=None),
):
    return {"ua": user_agent, "session": session_id}

@app.post("/set-session")
def set_session():
    resp = JSONResponse(content={"ok": True})
    resp.set_cookie(key="session_id", value="abc123", httponly=True)
    return resp
```

Phase 11 — FastAPI for GenAI Apps

This is where everything comes together. We'll build an LLM-powered backend using GPT-4o-mini for chat and text-embedding-3-small for embeddings — both fast and inexpensive, ideal for learning.

Install

```
(venv) $ pip install openai numpy
```

Add your key to .env:

```
# .env
OPENAI_API_KEY=sk-your-key-here
```

1. Simple chat endpoint

The most basic GenAI endpoint: receive a message, send to GPT-4o-mini, return the reply.

```
# app/routers/chat.py
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from openai import AsyncOpenAI
from app.config import settings

router = APIRouter(prefix="/ai", tags=["ai"])
client = AsyncOpenAI(api_key=settings.openai_api_key)

class ChatIn(BaseModel):
    message: str
    system: str = "You are a helpful assistant."

class ChatOut(BaseModel):
    reply: str
    model: str
    tokens_used: int

@router.post("/chat", response_model=ChatOut)
async def chat(payload: ChatIn):
    try:
        resp = await client.chat.completions.create(
```

```
            model="gpt-4o-mini",
            messages=[
                {"role": "system", "content": payload.system},
                {"role": "user", "content": payload.message},
            ],
        )
    except Exception as e:
        raise HTTPException(502, f"LLM error: {e}")

    return ChatOut(
        reply=resp.choices[0].message.content,
        model=resp.model,
        tokens_used=resp.usage.total_tokens,
    )
```

2. Multi-turn chat (conversation history)

```
from typing import Literal

class Message(BaseModel):
    role: Literal["user", "assistant", "system"]
    content: str

class ConversationIn(BaseModel):
    messages: list[Message]
    temperature: float = 0.7

@router.post("/chat/conversation")
async def conversation(payload: ConversationIn):
    resp = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[m.model_dump() for m in payload.messages],
        temperature=payload.temperature,
    )
    return {"reply": resp.choices[0].message.content}
```

3. Streaming chat — tokens as they arrive

For a real chat UI, you want tokens to appear progressively. Use FastAPI's StreamingResponse and OpenAI's streaming mode.

```
from fastapi.responses import StreamingResponse
```

```
@router.post("/chat/stream")
async def chat_stream(payload: ChatIn):
    async def token_generator():
        stream = await client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[
                {"role": "system", "content": payload.system},
                {"role": "user", "content": payload.message},
            ],
            stream=True,
        )
        async for chunk in stream:
            delta = chunk.choices[0].delta.content
            if delta:
                yield delta

    return StreamingResponse(token_generator(), media_type="text/plain")
```

4. Server-Sent Events (better for chat UIs)

```
@router.post("/chat/sse")
async def chat_sse(payload: ChatIn):
    async def event_stream():
        stream = await client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": payload.message}],
            stream=True,
        )
        async for chunk in stream:
            delta = chunk.choices[0].delta.content
            if delta:
                yield f"event: token\ndata: {delta}\n\n"
        yield "event: done\ndata: [DONE]\n\n"

    return StreamingResponse(event_stream(), media_type="text/event-stream")
```

5. WebSocket chat

```
from fastapi import WebSocket

@router.websocket("/ws/chat")
async def ws_chat(ws: WebSocket):
    await ws.accept()
```

```
    history = []
    try:
        while True:
            user_msg = await ws.receive_text()
            history.append({"role": "user", "content": user_msg})

            stream = await client.chat.completions.create(
                model="gpt-4o-mini",
                messages=history,
                stream=True,
            )
            full = ""
            async for chunk in stream:
                delta = chunk.choices[0].delta.content
                if delta:
                    full += delta
                    await ws.send_text(delta)
            history.append({"role": "assistant", "content": full})
            await ws.send_text("[END]")
    except Exception:
        await ws.close()
```

6. Embeddings endpoint

Embeddings turn text into a list of numbers (a vector). Similar texts produce similar vectors. This is the building block for semantic search and RAG.

```
class EmbedIn(BaseModel):
    text: str | list[str]

@router.post("/embed")
async def embed(payload: EmbedIn):
    inputs = payload.text if isinstance(payload.text, list) else [payload.text]
    resp = await client.embeddings.create(
        model="text-embedding-3-small",
        input=inputs,
    )
    return {
        "model": resp.model,
        "count": len(resp.data),
        "dim": len(resp.data[0].embedding),
        "vectors": [d.embedding for d in resp.data],
    }
```

7. Mini RAG — Retrieval-Augmented Generation

A complete RAG flow in one file: store documents with embeddings, then answer questions by retrieving the most similar documents and giving them to the LLM as context.

```
# app/routers/rag.py
import numpy as np
from fastapi import APIRouter
from pydantic import BaseModel
from openai import AsyncOpenAI
from app.config import settings

router = APIRouter(prefix="/rag", tags=["rag"])
client = AsyncOpenAI(api_key=settings.openai_api_key)

# Very simple in-memory store. In production: Pinecone, Qdrant, Chroma, pgvector.
STORE: list[dict] = []  # each entry: {"text": str, "vector": np.ndarray}

async def embed_one(text: str) -> np.ndarray:
    resp = await client.embeddings.create(
        model="text-embedding-3-small",
        input=text,
    )
    return np.array(resp.data[0].embedding)

def cosine(a, b):
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

class IngestIn(BaseModel):
    documents: list[str]

@router.post("/ingest")
async def ingest(payload: IngestIn):
    """Embed each document and store it."""
    for doc in payload.documents:
        vec = await embed_one(doc)
        STORE.append({"text": doc, "vector": vec})
    return {"stored": len(payload.documents), "total": len(STORE)}

class AskIn(BaseModel):
    question: str
    top_k: int = 3

@router.post("/ask")
async def ask(payload: AskIn):
    if not STORE:
```

```
    return {"answer": "No documents indexed yet.", "sources": []}

# 1) Embed the question
q_vec = await embed_one(payload.question)

# 2) Score every document by cosine similarity
scored = [(cosine(q_vec, d["vector"]), d["text"]) for d in STORE]
scored.sort(key=lambda x: x[0], reverse=True)
top = scored[: payload.top_k]

# 3) Build a context block and ask the LLM
context = "\n---\n".join(text for _, text in top)
prompt = (
    f"Use ONLY the context below to answer. "
    f"If the answer isn't there, say you don't know.\n\n"
    f"Context:\n{context}\n\nQuestion: {payload.question}"
)

resp = await client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}],
)
return {
    "answer": resp.choices[0].message.content,
    "sources": [{"score": round(s, 3), "text": t} for s, t in top],
}
```

Wire it into main.py:

```
# app/main.py
from app.routers import chat, rag
app.include_router(chat.router)
app.include_router(rag.router)
```

Test the RAG flow:

```
# 1. Ingest some documents
$ curl -X POST http://127.0.0.1:8000/rag/ingest \
  -H "Content-Type: application/json" \
  -d '{"documents":[
    "FastAPI uses Pydantic for data validation.",
    "Uvicorn is the ASGI server commonly used with FastAPI.",
    "GPT-4o-mini is OpenAI\"s small, fast, cheap chat model."
  ]}'
```

```
# 2. Ask a question
$ curl -X POST http://127.0.0.1:8000/rag/ask \
    -H "Content-Type: application/json" \
    -d '{"question":"What server runs FastAPI?"}'
```

8. File upload → ingest pipeline

```
@router.post("/ingest-file")
async def ingest_file(file: UploadFile = File(...)):
    text = (await file.read()).decode("utf-8")

    # Naive chunking — split into ~500-char pieces
    chunks = [text[i:i+500] for i in range(0, len(text), 500)]

    for chunk in chunks:
        vec = await embed_one(chunk)
        STORE.append({"text": chunk, "vector": vec})

    return {"chunks_added": len(chunks), "total": len(STORE)}
```

From this RAG to production

The in-memory STORE works for ~1000 documents. For real apps, replace it with a vector database — Qdrant, Chroma, Pinecone, or pgvector. The endpoint code barely changes; you just swap the storage layer.

9. Structured output with Pydantic

Force the LLM to return JSON that matches a Pydantic schema — useful for data extraction and tool calls.

```
import json

class Person(BaseModel):
    name: str
    age: int
    occupation: str

@router.post("/extract-person")
async def extract_person(text: str):
    resp = await client.chat.completions.create(
        model="gpt-4o-mini",
```

```
    messages=[
        {"role": "system", "content": "Extract a person from the text. Return JSON with name, age, occupation."},
        {"role": "user", "content": text},
    ],
    response_format={"type": "json_object"},
)
data = json.loads(resp.choices[0].message.content)
return Person(**data)   # Pydantic validates the LLM's output
```

Phase 12 — Deployment

Production server

Replace uvicorn --reload with a multi-worker setup.

```
(venv) $ pip install gunicorn
(venv) $ gunicorn app.main:app -k uvicorn.workers.UvicornWorker -w 4 -b 0.0.0.0:8000
```

Dockerfile

```
FROM python:3.12-slim

WORKDIR /code

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY ./app /code/app

CMD ["gunicorn", "app.main:app", \
    "-k", "uvicorn.workers.UvicornWorker", \
    "-w", "4", \
    "-b", "0.0.0.0:8000"]
```

Build and run:

```
$ docker build -t my-fastapi .
$ docker run -p 8000:8000 --env-file .env my-fastapi
```

Where to deploy

Platform	Best for
Render	Easiest. Push to GitHub, auto-deploy. Free tier available.
Railway	Similar to Render. Great DX.
Fly.io	Global edge, generous free tier, Docker-native.

Platform	Best for
AWS / GCP / Azure	Full control, more complex. Use for scale.
Vercel / Netlify	Limited — they're serverless. Streaming and WebSockets need workarounds.

Environment variables in production
Never bake API keys into your Docker image. Pass them at runtime: docker run --env-file .env ..., or set them in your hosting platform's dashboard.

Phase 13 — Production Concepts

Logging

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
)
log = logging.getLogger("app")

@app.post("/chat")
async def chat(payload: ChatIn):
    log.info("chat request, model=gpt-4o-mini")
    ...
```

Rate limiting with slowapi

```
(venv) $ pip install slowapi
```

```
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

@app.get("/expensive")
@limiter.limit("10/minute")
async def expensive(request: Request): ...
```

API versioning

```
from fastapi import APIRouter

v1 = APIRouter(prefix="/v1")
```

```
v1.include_router(chat.router)
v1.include_router(rag.router)

app.include_router(v1)

# Now everything lives under /v1/...
# Add /v2 later without breaking existing clients.
```

Health and readiness checks

```
@app.get("/healthz")  # liveness — am I alive?
def healthz():
    return {"ok": True}

@app.get("/readyz")   # readiness — am I ready to serve traffic?
async def readyz(db: Session = Depends(get_db)):
    db.execute("SELECT 1")
    return {"ready": True}
```

Testing

```
(venv) $ pip install pytest httpx
```

```
# tests/test_chat.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_home():
    r = client.get("/")
    assert r.status_code == 200

def test_create_note():
    r = client.post("/notes", json={"title": "t", "content": "c"})
    assert r.status_code == 201
    assert r.json()["title"] == "t"
```

```
(venv) $ pytest
```

Cheat Sheet

Decorators

Pattern	Use
@app.get('/x')	Read endpoint
@app.post('/x', status_code=201)	Create endpoint
@app.put('/x/{id}')	Replace
@app.patch('/x/{id}')	Partial update
@app.delete('/x/{id}', status_code=204)	Delete
@app.websocket('/ws')	WebSocket
@app.middleware('http')	Middleware
@app.exception_handler(MyError)	Global error handler

Common imports

```
from fastapi import (
    FastAPI, APIRouter, Depends, HTTPException, status,
    Query, Path, Body, Header, Cookie, Form, File, UploadFile,
    BackgroundTasks, Request, WebSocket,
)
from fastapi.responses import (
    JSONResponse, StreamingResponse, FileResponse, HTMLResponse,
)
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from pydantic import BaseModel, Field, EmailStr
```

Frontend snippets (browser-side)

```
// Plain JSON POST
const r = await fetch("/ai/chat", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ message: "Hello" }),
```

```
});
const data = await r.json();

// Consume a streaming endpoint
const r = await fetch("/ai/chat/stream", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ message: "Tell me a joke" }),
});
const reader = r.body.getReader();
const decoder = new TextDecoder();
while (true) {
  const { value, done } = await reader.read();
  if (done) break;
  console.log(decoder.decode(value));   // each chunk = some tokens
}

// WebSocket
const ws = new WebSocket("ws://localhost:8000/ai/ws/chat");
ws.onmessage = e => console.log(e.data);
ws.onopen = () => ws.send("Hello");
```

What's Next

You now have the full FastAPI toolkit: setup, endpoints, validation, database, auth, async, file handling, and a working GenAI backend with chat, streaming, and RAG.

Practical next steps to lock in the knowledge:

1. **Build a Chat-with-PDF app.** Upload a PDF, chunk it, embed each chunk, then answer questions about it.
2. **Add conversation persistence.** Store chat history in the database, link it to authenticated users.
3. **Build a tool-calling agent.** Let GPT-4o-mini call your own FastAPI endpoints as tools.
4. **Connect a frontend.** Use Next.js or Vite + React to consume your streaming endpoint with the EventSource API.
5. **Swap in a real vector DB.** Replace the in-memory STORE with Qdrant or pgvector.
6. **Deploy it.** Push to Render or Fly.io, point a domain at it, and share it with someone.

Build something. Ship it. Iterate.